Lab 3: Clean new data

Today we cleaned a messy table step by step: text, types, dates and duplicates. Cleaning is something you learn by doing it, and the best test is a table you have **not** seen before. So in this lab you get new data: the loans of a library. **The library and its data are made up**, but the problems in the table are of the kinds that real data has. This lab has two parts, and **you choose**.

Part 1: Practise repeats the steps from today on the new table. The problems are the same kinds as today, in new places. Do this part if you want to try the whole routine once more on your own.

Part 2: Go deeper is for you who felt "I understood this, and I want to go further". The questions are more open, and you have to decide what is right when the data is not clear.

I do not expect you to do both parts. It is your own choice, and you are welcome to contact me if you want to talk about which part to take.

Download **loans.csv** from Teams and save it in your `data` folder. Work in a new notebook. Always keep the original: load it into `loans_raw`, and clean a copy that you call `loans`. Write a short comment above each step, in your own words. If you get stuck: read the error message from the bottom up, print the thing you are working with, and look at the lecture notebook. Then ask.

Part 1: Practise

Clean the new table with the same steps as today. Same kinds of problems, new places.

Tools you need

```
loans_raw = pd.read_csv('data/loans.csv')      loans = loans_raw.copy()
loans['x'].nunique()        loans['x'].value_counts()
loans['x'].str.strip().str.lower()        .str.title()
loans['x'].replace({'old': 'new'})         # whole values
loans['x'].str.replace('kr', '')           # part of the text
loans['x'].map({'yes': True, 'no': False}) # unknown values become NaN
loans['x'].astype(float)
pd.to_datetime(loans['x'], errors='coerce')    loans['x'].dt.to_period('M')
loans.duplicated().sum()     loans.duplicated(subset='x').sum()
loans.drop_duplicates().reset_index(drop=True)      loans.drop(columns='x')
loans.to_csv('data/loans_clean.csv', index=False)
```

Tasks

1 **Load and look**
Load the file into `loans_raw` and make the copy `loans`. Print the shape and run `info()`. Write down in a comment **every problem you can see already**: which columns are text that should be something else, and which column is empty?

2 **Clean the genre**
How many different values does `genre` have? It should be six genres. Remove spaces and capital letters with `str.strip()` and `str.lower()` and count again. Look at the values that are left with `unique()` and fix the rest with `replace`. Finish with `str.title()`. How many genres do you have, and how many loans in each?

LEXICON

3 **Yes and no**

The column `returned` says whether the book is back. How many different values does it have? Turn it into `True` and `False` with `strip`, `lower` and `map`. Check that **nothing** became missing, and count `True` and `False`.

4 **The late fee is text**

The late fee looks like `7,50 kr`. Turn it into a number. First try `astype(float)` directly and read the error. Then remove `kr`, remove the spaces, change the comma to a dot, and only then use `astype(float)`. Check the result with `describe()` and calculate the total of all fees.

On purpose: Try `astype(float)` on the original text first, and read the error from the bottom up.

5 **Dates**

Turn `loan_date` into real dates with `pd.to_datetime(..., errors='coerce')`. How many dates became `NaT`? Find the first and the last date. Then count the loans per month with `dt.to_period('M')`, and check if the numbers add up to the whole table.

6 **Duplicates**

How many duplicates are there in the **raw** table, and how many after the cleaning? Why is the number different? Check with `subset='loan_id'`. Remove the duplicates, and reset the index. How much did the duplicates add to the total of the late fees, in kr and in percent?

7 **Put it together**

Remove the empty column `notes`. Then collect all your steps in **one cell** that starts from `loans_raw` and gives the finished table. Run it, check the shape, and save the table as `data/loans_clean.csv`. Run the cell twice. Does it work both times? Why?

Expected

Task 1: (623, 9). The column `notes` is completely empty. `loan_date`, `return_date` and `late_fee` are text, and `returned` is text. `return_date` has 593 values (30 are missing).
Task 2: 28 values at first, then 9 after `strip` and `lower`. After the fix and `title()`: 6 genres. Crime 163, Children 135, Fantasy 90, Sci-Fi 90, Romance 86, History 59.
Task 3: 10 different values at first. After the fix: True 593 and False 30, with 0 missing.
Task 4: `ValueError: could not convert string to float: '7,50 kr'`. After the fix: min 0.0, max 47.5, mean 12.32, and a total of 7677.5.
Task 5: 3 dates become `NaT`. First date 2026-01-05, last date 2026-06-27. Per month: 96, 100, 111, 106, 108 and 99. They add up to 620, which is 623 minus the 3 `NaT`.
Task 6: 12 duplicates in the raw table and 18 after the cleaning (6 more, which differ only in spelling). `subset='loan_id'` also gives 18. After removing: (605, 9). The total fee goes from 7677.5 to 7447.5, so the duplicates added 230.0 kr, about 3.0 percent.
Task 7: (605, 8). The cell gives the same result the second time, since it starts from `loans_raw` every time.

LEXICON

Part 2: Go deeper

Here you meet problems that need a decision, and you write your own function. There is one new tool (`dt.days`), and it is in the box below.

Tools you need

```
(loans['b'] - loans['a']).dt.days                    # days between two dates
loans[loans['b'] < loans['a']]                       # rows where b is before a
loans.duplicated(subset=['member_id', 'title', 'loan_date'], keep=False)
def clean_loans(raw):
    table = raw.copy()
    ...                                              # your steps
    return table
```

Tasks

1 **Missing and impossible return dates**

Add `return_date` to your one-cell cleaning from Part 1 (turn it into dates with `errors='coerce'`, before you remove the duplicates) and run the cell again. How many return dates are missing? Compare with the number of loans where `returned` is `False`. What does that tell you about **why** they are missing?

Then find the loans where the return date is **before** the loan date. How many are there? Look at them and decide what to do with them. Write your decision in a comment, and what it does to the numbers. Calculate the number of days each loan lasted with `(return_date - loan_date).dt.days` and see the minimum.

---

2 **Is it the same loan?**

Some loans look the same but have different `loan_id`: same member, same book, same loan date. Find them with `duplicated(subset=['member_id', 'title', 'loan_date'], keep=False)`. How many rows are there, and what is different between the two rows in each pair?

Are they the same loan that was registered twice, or two real loans? You can argue both ways. Decide, write your reasons in a comment, and calculate what your decision does to the total of the late fees.

---

3 **Make it reusable**

Write a function `clean_loans(raw)` that takes the raw table and returns the finished table, with all the steps from Part 1 and Part 2 (the genre, `returned`, the late fee, `loan_date`, `return_date`, duplicates and `notes`). Test it on `loans_raw`. Check the shape, the total of the fees and the number of missing return dates. Why is a function better than a long list of cells?

---

4 **Fees by genre**

Use your table **without** the pairs from task 2 (or explain if you keep them). For every genre, calculate the number of loans, the share of loans with a late fee above 0, and the average late fee. Which genre has the highest share of late loans? Write what you **can** and **cannot** conclude.



5 Your own question

Come up with **one question** about the library data that you can answer with the tools from today. Write the question as a comment, answer it with code, and finish with a comment about what the answer does **not** tell you. That last part is the most important one.

Expected

Task 1: 30 return dates are missing, and 30 loans have `returned` equal to `False`: the book is not back yet, so there is no return date. That is missing **for a reason**. 6 loans have a return date before the loan date, with a shortest length of -5 days. Mean length without them: 22.28 days.
Task 2: 10 rows, 5 pairs, e.g. member M0023 and "The Locksmith" on 2026-03-15. The `loan_id` is the only difference (L-30238 and L-40004). If you remove one of each pair, the total fee goes from 7447.5 to 7362.5 (85.0 kr less) and the table has 600 rows.
Task 3: the function gives (605, 8), a total fee of 7447.5 and 30 missing return dates, the same as the step by step result.
Task 4: Sci-Fi has the highest share of late loans (0.539, 89 loans) and Fantasy the lowest (0.448, 87 loans).

Stuck? Read the error message from the bottom up, print the data you are working with, and then ask.